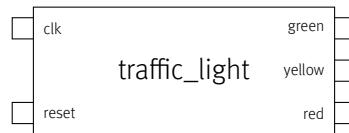


Traffic Light Controller

In this introductory task you will design a traffic light controller. The interface of the module is depicted in the following figure:



The clk input is mapped to the clock, the reset input is an active low reset, so the module should reset itself if this input is low. The module has three outputs red, yellow and green which are connected to the respective light. If an output is 1, then the light is on. If it is 0, then the light is off.

The traffic light controller should make use of a finite state machine (FSM). The traffic light will have four states: Red (R), RedYellow (RY), Green (G) and Yellow (Y). The transitions between these states are controlled by the clock and reset signals. The functionality of the traffic light controller is as follows:

- The traffic light shows a red light in the beginning. The state should be set to red whenever reset is low
- After reset goes to high, the traffic light should start its operation
- After one cycle, the red and yellow light should be illuminated. One cycle later only green should be lit. One cycle later, it should switch to yellow
- This finishes one full cycle of the traffic light. After the yellow light, it should switch back to red and start over. The whole sequence can be seen in Figure 1
- As long as reset is high, it should loop forever with $R \rightarrow RY \rightarrow G \rightarrow Y \dots$

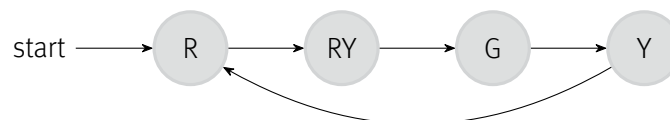


Figure 1: State diagram of the traffic light controller FSM. The states are Red (R), Red and Yellow (RY), Green (G) and Yellow (Y). The reset signal is omitted to simplify the diagram.

Your tasks

a) In your repository you can find the module definition of the traffic light module, the corresponding testbench and a Makefile. Implement the behavior of the module as described above. You can test your implementation by running the testbench (»make simulation«) and looking at the terminal output.

If you want to debug, then inspect your module using a waveform viewer (for example by running »surfer traffic_light.vcd«). Compare your solution with the timing diagram in Figure 2. What happens if the reset is asynchronously activated?

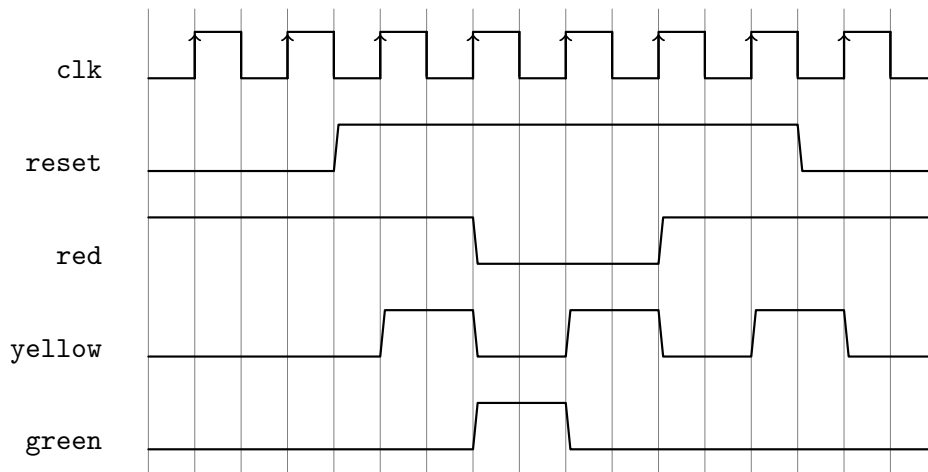


Figure 2: Timing diagram of the traffic light controller. The inputs of the module are the clk and reset signals. The outputs are the red, yellow and green signals controlling the traffic lights. The output are change according to the state diagram in Figure 1.

b) Flash the traffic light controller on the FPGA and connect the outputs to the LEDs on the board. For this you just have to run »make flash_bitstream«. The mappings between SystemVerilog signals and FPGA pins are defined in the file »traffic_light.pcf«. All the mappings start with `set_io -nowarn` followed by the name in SystemVerilog and lastly the number of the I/O port on the FPGA. You can find the corresponding numbers on the pinout of the pico-ice (see Figure 3).

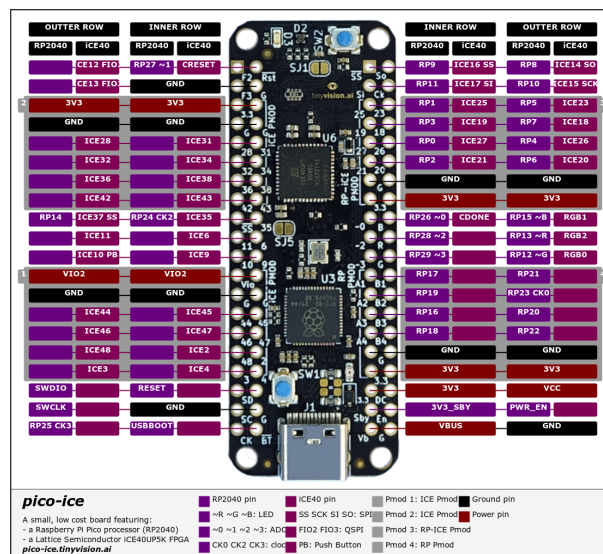


Figure 3: Pinout of the pico-ice FPGA

As a note: Some ports cannot be mapped to the FPGA, they are exclusive to the RaspberryPi. All the possible I/O ports start with »ICE«. Also, some ports have a fixed mapping. E.g. pin ICE35 is the 12 MHz system clock and the button further away from the USB port is mapped to port ICE10.

Check the port mapping in the pcf file or define your own ports and then connect LEDs to your outputs. LEDs have a long leg and a short leg. The longer leg is the anode and should be connected to your pins for red, yellow and green (they deliver 3.3 V). The shorter leg is the cathode and it has to be connected in series with a resistor to ground (labelled GND in Figure3). Resistors with 47 Ω or 100 Ω are fine. You can find a box of different resistors in the lab. If you are completely unsure on what to do, then you can read more about breadboards and LEDs here.

You should now see all LEDs glowing simultaneously at a low brightness. Why does this behavior seem different from the waveform you saw in a)? How can you fix this?

Hint: How many clock cycles occur in one second if the speed of the clock is 12 MHz? Make use of this information to stay longer in each state. If you want to simulate your changes you probably want to adjust the simulated timespan in the testbench. For example: you can simulate 5 time units more by adding #5 in the test procedure.

c) Change the duration of each state: The red and green states should be on for 6 s respectively. Because our traffic light controller will be installed inside the city, the yellow state should on for 3 s according to legislation. The RY state is 2 s long. Modify your design accordingly.

d) Bonus task Most traffic lights for pedestrians make a sound in the green phase to help blind people cross the street safely. You can find speakers in the big red box with the resistors. Extend your traffic light controller with the ability to play a sound in the green phase.

Hint: You need to define a new output of your module that you connect to the positive wire (red) of the speaker. Connect the other wire (black) to ground. The speaker simply needs a sine wave across its pins to work. We cannot produce a proper sine wave with digital outputs, but the rectangle shaped clock is still good enough. Which frequencies do you need for the speaker? How can you generate a clock with your desired frequency?